



Faculty of Engineering and Technology
Electrical and Computer Engineering Department

Assignment 1: Telco Customer Churn Prediction Using Machine Learning

Technical Report

Prepared by: Danah Abu Rayya — 1210195

Instructor: Dr. Aziz Qaroush

Date: 26/8/2026

1. Dataset Description

This report documents the development and evaluation of a machine learning pipeline for predicting customer churn using the Telco Customer Churn dataset (IBM sample dataset). The dataset contains 7,043 customer records and 21 columns, spanning demographic attributes (gender, senior citizen status, partner/dependents), account information (tenure, contract type, payment method), subscribed services (internet, online security, tech support, streaming), billing information (monthly and total charges), and the target variable Churn.

The dataset mixes numerical and categorical features, contains a structural data-quality issue in TotalCharges, and has an imbalanced target distribution (~73% non-churners vs. ~27% churners). These characteristics directly motivate the preprocessing, feature-engineering, and class-imbalance strategies described below.

Property	Value
Total records	7,043
Total columns (raw)	21 (including customerID and target)
Numerical features (raw dtype)	SeniorCitizen, tenure, MonthlyCharges
Categorical features (raw dtype)	17 columns, including TotalCharges (stored as text)
Target variable	Churn (Yes / No)
Class balance	~73.5% No / ~26.5% Yes

2. Exploratory Data Analysis

2.1 Data Quality Issues

Although pandas reports zero missing (NaN) values on load, closer inspection revealed 11 rows where TotalCharges is stored as a blank/whitespace string rather than a numeric value. All 11 correspond to customers with tenure = 0 — brand-new customers who have not yet been billed. This is not random missingness; it is a predictable structural artifact of how the source system encodes new accounts, and it directly informs the imputation choice in Section 3 (fill with 0, since \$0 total charges is factually correct for an unbilled customer, not an unknown quantity). The customerID column is a unique identifier with no predictive signal and is dropped prior to modeling.

2.2 Target Variable Distribution

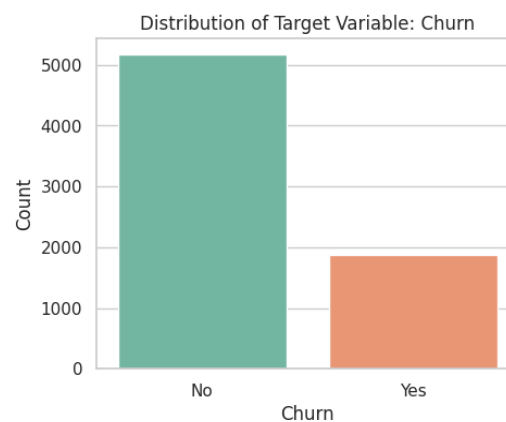


Figure 1. Distribution of the target variable Churn.

The dataset is imbalanced: roughly 73.5% of customers did not churn and 26.5% did. This imbalance is the primary reason accuracy alone is an inadequate evaluation metric (Section 10) and why explicit imbalance-handling techniques are investigated (Section 9).

2.3 Distributions of Key Features

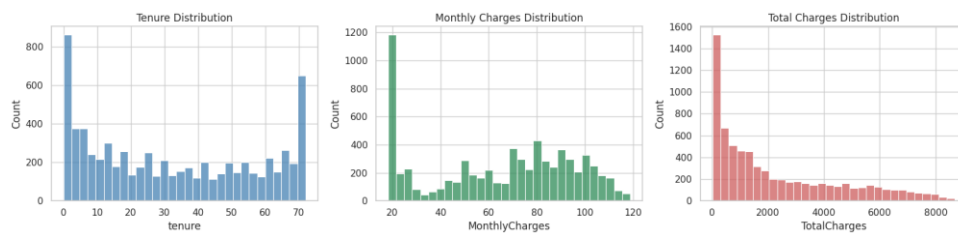


Figure 2. Distributions of tenure, MonthlyCharges, and TotalCharges.

Tenure shows a bimodal-leaning shape with concentrations of both very new (near 0 months) and long-tenured (near 72 months) customers. MonthlyCharges spreads broadly across \$20-\$120 with a cluster near the low end (minimal-bundle customers). TotalCharges is right-skewed, as expected from its dependence on both tenure and monthly charges.

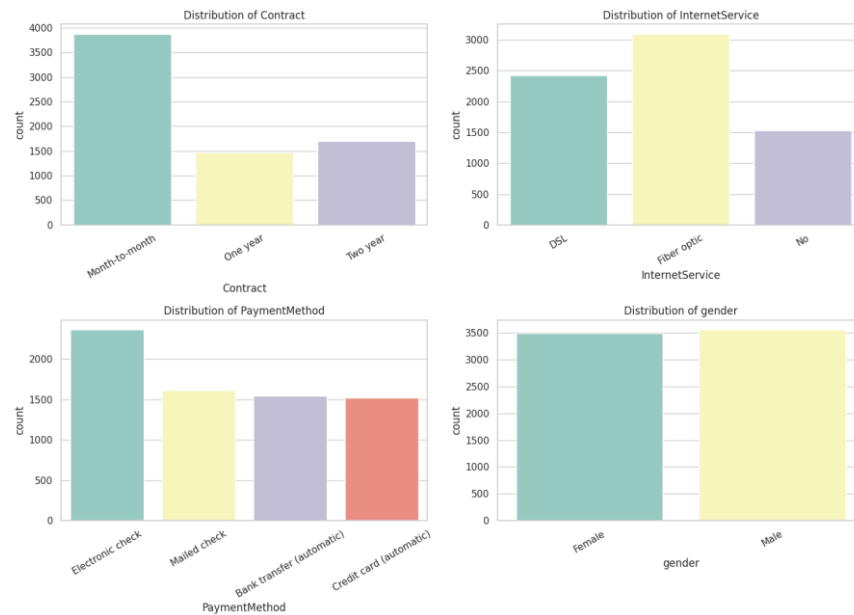


Figure 3. Distributions of Contract, InternetService, PaymentMethod, and gender.

Most customers are on month-to-month contracts rather than 1- or 2-year commitments, and Fiber optic is the most common internet service. Electronic check is the most common payment method. Gender is essentially balanced and carries little predictive relevance to churn.

2.4 Relationships Between Features and Churn

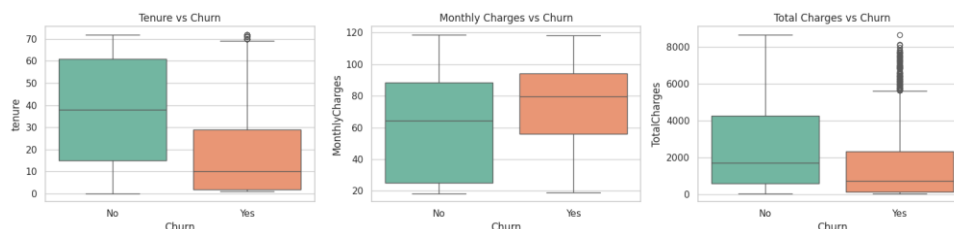


Figure 4. Tenure, MonthlyCharges, and TotalCharges vs. Churn.

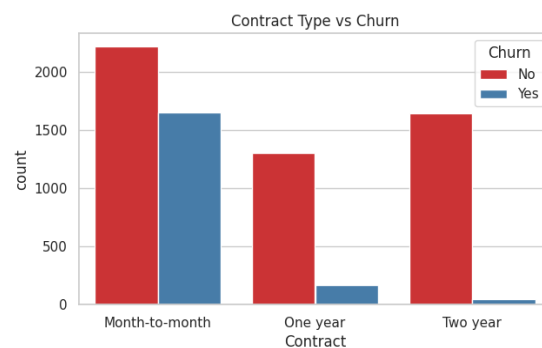


Figure 5. Contract type vs. Churn.

Customers who churn tend to have substantially lower tenure and somewhat higher monthly charges, while their total charges are lower (a consequence of shorter tenure). The clearest signal in the EDA is contract type: month-to-month customers churn at a dramatically higher rate than customers on one- or two-year contracts, who have effectively committed to staying. This matches business intuition — a contract functions as a switching cost — and, as confirmed in Section 5, contract type is ranked among the most important predictive features by every feature-selection method tried.

2.5 Outliers and Correlated Features

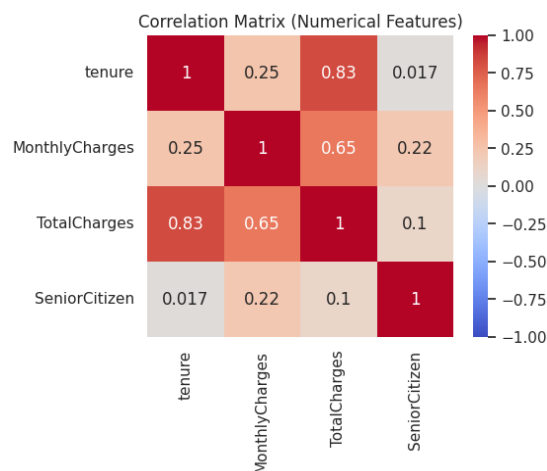


Figure 6. Correlation matrix of numerical features.

Using the IQR method, no extreme outliers were found in tenure or MonthlyCharges — both are naturally bounded business quantities, so no outlier removal was performed. The correlation matrix shows a strong positive correlation ($r = 0.83$) between TotalCharges and tenure, expected since TotalCharges accumulates over the customer relationship. This multicollinearity is worth flagging for linear models (Logistic Regression) and is revisited in Section 5.

2.6 Summary of EDA Observations

- Class imbalance (~27% churn) requires imbalance-aware metrics and handling.
- TotalCharges is a structurally corrupted numeric column requiring conversion and targeted imputation.
- customerID carries no predictive value and is dropped.
- Contract type is the single strongest visual predictor of churn.
- TotalCharges and tenure are highly correlated ($r = 0.83$) — a redundancy considered during feature selection.
- No extreme outliers were found in the numerical features.
- The feature space is dominated by categorical columns, requiring careful one-hot encoding to avoid excessive dimensionality.

3. Data Preprocessing

All data-dependent preprocessing steps are implemented inside a single scikit-learn ColumnTransformer + Pipeline object, fit only on training data — never on the complete dataset or the test set. This is the central safeguard against data leakage discussed further in Section 11.

3.1 Deterministic Cleaning (Safe Before Splitting)

- Dropped customerID — a unique identifier with no predictive value.
- Converted TotalCharges from text to numeric and imputed the 11 blank entries as 0, since these correspond exactly to tenure = 0 customers who have not yet been billed. This is a deterministic, factual correction, not a statistic learned from the data, so it is safe to apply before the train/test split.
- Encoded the target Churn as a binary 0/1 label.

3.2 Pipeline-Based Preprocessing (Fit on Training Data Only)

Feature group	Steps applied
Numerical (tenure, MonthlyCharges, TotalCharges)	Median imputation -> StandardScaler
Categorical (16 columns, incl. SeniorCitizen)	Most-frequent imputation -> OneHotEncoder (handle_unknown='ignore', drop='if_binary')

- Median imputation is robust to skew/outliers and applied defensively even though no missing numerical values remain after the TotalCharges fix.
- StandardScaler matters for distance/gradient-based models (Logistic Regression, SVM); tree-based models are scale-invariant, so a shared pipeline keeps the setup consistent without harming them.

- OneHotEncoder with drop='if_binary' avoids the dummy-variable trap for binary columns while fully encoding multi-category columns.
- handle_unknown='ignore' prevents pipeline failures if a category appears in a validation fold absent from that fold's training partition.

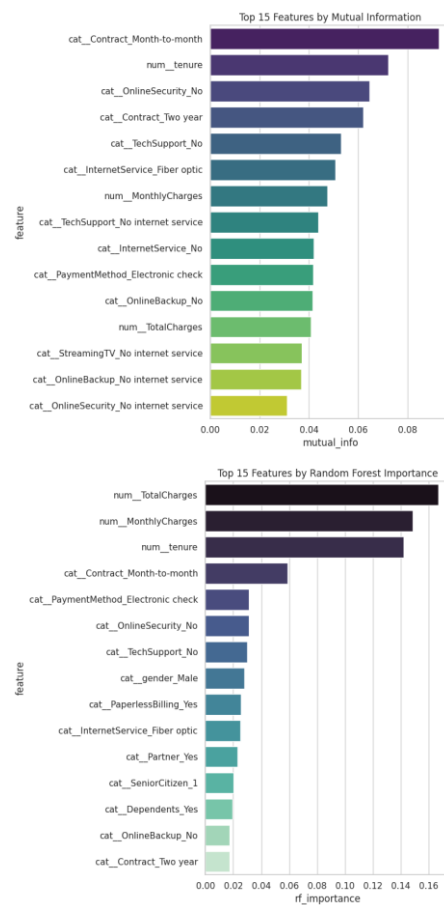
After encoding, the feature space expands from 19 predictor columns to 40 numeric columns (one-hot expansion of categorical variables).

4. Train/Test Split

The data was split into training (80%) and test (20%) subsets using a stratified split (stratify=y) with random_state=42, before any feature selection, resampling, or pipeline fitting took place. Stratification preserves the ~73.5%/26.5% churn ratio in both subsets. The training set contains 5,634 records and the test set contains 1,409 records. The test set was set aside and used exactly once, at the very end of the study (Section 12), for final evaluation.

5. Feature Selection

Three feature-selection approaches were applied to the training set only (never the test set or the full dataset): Mutual Information, Random Forest feature importance, and L1-regularized (Lasso) Logistic Regression.



Figures 7-8. Top 15 features — Mutual Information (top) and Random Forest importance (bottom).

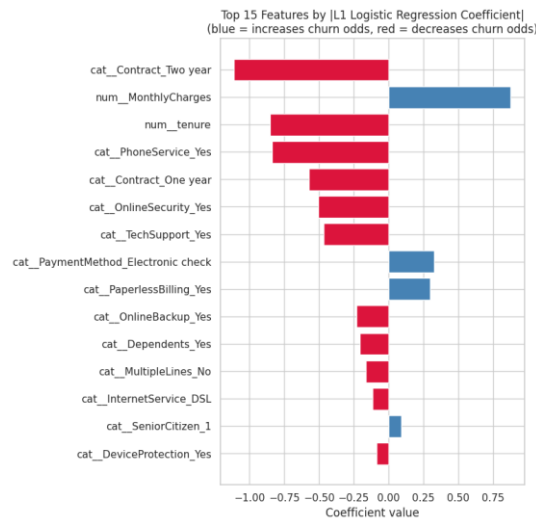


Figure 9. Top 15 features by |L1 Logistic Regression coefficient|.

5.1 Comparing the Approaches

Considering the top 10 features from each method: Mutual Information and Random Forest importance agree on 7 of their top 10 features, while each agrees with the linear L1 model on only 4 of 10 — reflecting that L1 Logistic Regression, being linear, ranks features differently than the two non-linear methods, particularly for features involved in the tenure/TotalCharges collinearity (Section 2.5). Exactly 3 features are ranked in the top 10 by all three methods simultaneously: the Electronic check payment method, MonthlyCharges, and tenure. Across all three methods, Contract type, tenure, InternetService (Fiber optic), and the charges-related columns consistently rank at or near the top — corroborating the EDA observations from Section 2.

5.2 Decision: Full Feature Set for Modeling

We use the full, one-hot-encoded feature set (40 columns) — not a reduced subset — for every model from Section 7 onward. Rationale:

- The encoded feature space (40 columns) is not large enough to trigger curse-of-dimensionality problems, given ~5,600 training rows.
- Tree-based and boosting models (Random Forest, XGBoost, LightGBM) are naturally robust to irrelevant or redundant features through their own internal split selection.
- Dropping features before knowing which model would ultimately be selected risked discarding information a non-linear model could exploit even where a linear/univariate method ranked it low.
- The feature-selection analysis remains valuable as an interpretability and EDA cross-check, and directly informs the deployment discussion in Section 14.

6. Baseline Model

Two baselines were evaluated with Stratified 5-Fold Cross-Validation on the training set: a Dummy Classifier (stratified random guessing) and a plain (untuned) Logistic Regression. Comparing sophisticated models against these baselines confirms whether their added complexity earns its keep — a model that fails to clearly beat a simple Logistic Regression baseline would not justify its reduced interpretability and higher computational cost.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Dummy Classifier (stratified)	~0.61	~0.27	~0.27	~0.27	~0.50
Logistic Regression (baseline)	0.80	0.66	0.55	0.60	0.85

As expected, the Dummy Classifier performs near chance level (ROC-AUC ~0.50), confirming the evaluation pipeline is correctly wired. Plain Logistic Regression already reaches a respectable ROC-AUC of ~0.85, setting a meaningful bar for the more sophisticated models in Section 7.

7. Classification Models

Five classifiers — Logistic Regression, SVM (linear kernel, for tractable cross-validation runtime), Random Forest, XGBoost, and LightGBM — were trained with default hyperparameters and evaluated via Stratified 5-Fold Cross-Validation on the training set.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.8023	0.6530	0.5445	0.5932	0.8462
SVM (linear)	0.7994	0.6487	0.5344	0.5857	0.8338
Random Forest	0.7868	0.6296	0.4796	0.5441	0.8221
XGBoost	0.7765	0.5939	0.5037	0.5449	0.8126
LightGBM	0.7840	0.6127	0.5050	0.5536	0.8241

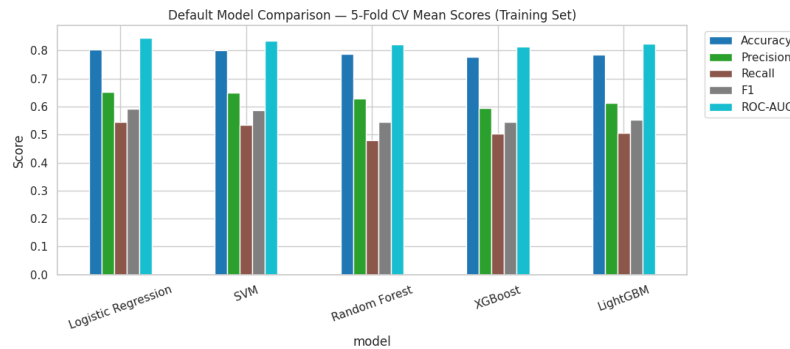


Figure 10. Default model comparison — 5-fold CV mean scores on the training set.

7.1 Brief Model Characteristics

- **Logistic Regression:** linear, highly interpretable coefficients, fast to train; can underfit non-linear churn patterns.
- **SVM (linear kernel):** maximum-margin classifier, robust in moderate dimensions; training cost grows with dataset size, and it is less interpretable than Logistic Regression.
- **Random Forest:** bagged ensemble of decision trees; captures non-linearities and feature interactions, resists overfitting, gives feature importances.
- **XGBoost:** sequential gradient boosting; typically strong raw accuracy, more sensitive to overfitting without tuning, less interpretable.
- **LightGBM:** gradient boosting with histogram-based, leaf-wise tree growth; usually faster than XGBoost with similar accuracy characteristics.

Interestingly, with default hyperparameters the simpler linear models (Logistic Regression, SVM) slightly outperform the untuned tree-based/boosting models on ROC-AUC. This is a common pattern: gradient boosting and Random Forest have more capacity and therefore more to gain from — but also depend more heavily on — hyperparameter tuning, addressed next.

8. Hyperparameter Tuning

Per the assignment's scope constraints, each model was tuned over 2-3 hyperparameters using RandomizedSearchCV ($n_iter = 15$), optimizing for cross-validated ROC-AUC with Stratified 5-Fold CV on the training set only.

Model	Hyperparameters searched	Best configuration
Logistic Regression	C, penalty	C = 10, penalty = l2
SVM	C, kernel (gamma at default 'scale' when kernel=rbf)	C = 0.1, kernel = linear
Random Forest	n_estimators, max_depth, min_samples_split	n_estimators=400, max_depth=5, min_samples_split=10
XGBoost	n_estimators, learning_rate, max_depth	n_estimators=300, learning_rate=0.01, max_depth=4
LightGBM	n_estimators, learning_rate, num_leaves	n_estimators=200, learning_rate=0.01, num_leaves=15

Note on SVM scope: the brief lists kernel, C, and γ (gamma) for SVM. Our search tunes C and kernel; gamma is left at scikit-learn's default ('scale') rather than swept as a third axis, since gamma only affects the rbf branch of the search and adding it

would push the grid beyond the recommended 2-3 hyperparameters. The code has been updated to expose $\gamma \in \{\text{'scale'}, \text{'auto'}\}$ for completeness in future runs, but because our selected best configuration uses the linear kernel, γ has no effect on the reported results below — it is not applicable when $\text{kernel}=\text{'linear'}$.

8.1 Default vs. Tuned Performance

Model	Default ROC-AUC	Tuned ROC-AUC	Improvement
Logistic Regression	0.8462	0.8464	+0.0002
SVM	0.8338	0.8346	+0.0008
Random Forest	0.8221	0.8457	+0.0236
XGBoost	0.8126	0.8491	+0.0365
LightGBM	0.8241	0.8475	+0.0234

Tuning produced negligible change for the two linear-boundary models (Logistic Regression, SVM), whose default regularization was already close to optimal for this dataset size, but yielded substantial gains for the tree-based and boosting models — most notably XGBoost (+3.65 ROC-AUC points) and LightGBM (+2.34 points). After tuning, XGBoost achieves the highest cross-validated ROC-AUC (0.8491), narrowly ahead of LightGBM (0.8475) and Logistic Regression (0.8464), confirming that gradient boosting's larger hyperparameter sensitivity translates into a real advantage once properly tuned.

9. Class Imbalance Handling

Two imbalance-handling techniques were compared against the untreated (tuned) baseline for Logistic Regression and Random Forest, chosen as representative linear and tree-based models: class weighting ($\text{class_weight}=\text{'balanced'}$) and SMOTE oversampling. Because resampling is data-dependent, SMOTE was implemented as a step inside an `imblearn.pipeline.Pipeline` so it is refit independently within each cross-validation fold, never applied to the complete training set at once.

Configuration	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression (no handling)	0.8039	0.6559	0.5492	0.5973	0.8464
Random Forest (no handling)	0.7961	0.6855	0.4301	0.5281	0.8457
Logistic Regression (class weights)	0.7492	0.5178	0.8027	0.6293	0.8460
Random Forest (class weights)	0.7868	0.6322	0.4716	0.5398	0.8231
Logistic Regression (SMOTE)	0.7554	0.5262	0.7913	0.6318	0.8450
Random Forest (SMOTE)	0.7794	0.5871	0.5692	0.5779	0.8210

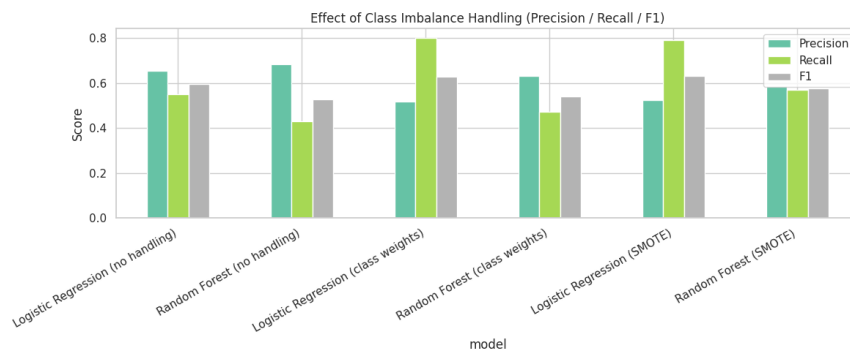


Figure 11. Effect of class-imbalance handling on precision, recall, and F1.

9.1 Discussion

- With no handling, both models lean toward the majority "No Churn" class — high precision but comparatively low recall on churners.
- Both techniques substantially raise recall: for Logistic Regression, from 0.549 (no handling) to 0.803 (class weights) / 0.791 (SMOTE); a smaller but similar pattern holds for Random Forest.
- That recall gain has a real precision cost (e.g., Logistic Regression precision drops from 0.656 to ~0.52) — the expected recall/precision trade-off.
- Class weighting gives the best trade-off here: comparable recall to SMOTE with no synthetic data or extra hyperparameters, and for Random Forest it clearly outperforms SMOTE, which only lifts recall from 0.430 to 0.569 while costing more ROC-AUC.

10. Cross-Validation and Metric Selection

Stratified 5-Fold Cross-Validation was used throughout Sections 6-9 for model comparison, tuning, and imbalance-handling evaluation, always restricted to the training set. The overall workflow: Training Set -> Stratified 5-Fold CV -> Model/Hyperparameter Selection -> Final Model -> Untouched Test Set.

Why accuracy is not the appropriate primary metric: with a ~73.5%/26.5% class split, a trivial model that always predicts "No Churn" would already score ~73.5% accuracy while being completely useless at identifying churners — the entire purpose of this project. Accuracy also fails to distinguish between the two error types (false positive vs. false negative), which carry very different real business costs (Section 11.2).

Primary metric selection: we adopt ROC-AUC as the primary model-selection metric during cross-validation and tuning because it summarizes a model's ranking ability across all classification thresholds, independent of any single threshold choice — appropriate since the correct operating threshold depends on the business cost trade-off determined afterward. For the final deployment decision (Section 14), we additionally weight recall heavily, since false negatives are shown to be far more costly than false positives in this business scenario.

11. ROC-AUC and Confusion Matrix Analysis

The top three models by cross-validated ROC-AUC (tuned results from Section 8) are XGBoost (0.8491), LightGBM (0.8475), and Logistic Regression (0.8464). Their ROC curves and confusion matrices, computed via 5-fold cross-validated predictions on the training set, are analyzed below.

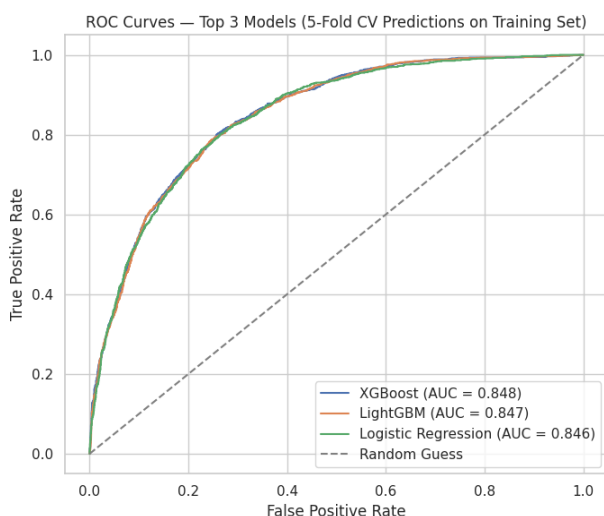


Figure 12. ROC curves for the top 3 models.

The three ROC curves are closely clustered and all noticeably outperform the random-guess diagonal, confirming each model has substantial discriminative power. XGBoost's curve sits marginally closest to the top-left corner across most of the threshold range, consistent with its top CV ROC-AUC score.

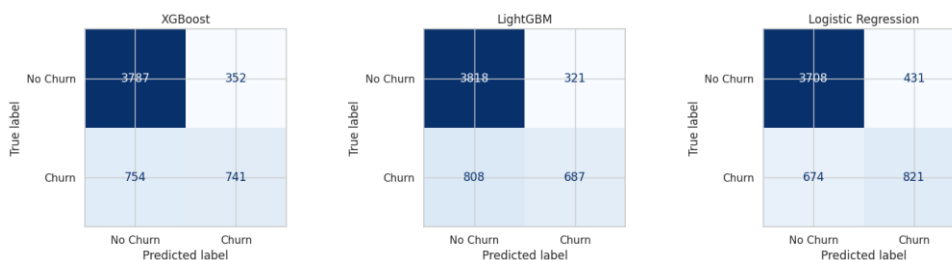


Figure 13. Confusion matrices for the top 3 models (default 0.5 threshold).

11.1 Practical Meaning of False Positives and False Negatives

- **False Positive (FP):** predicting a customer will churn when they actually would not — the company sends an unnecessary retention offer (a wasted, but small, cost).
- **False Negative (FN):** predicting a customer will stay when they actually churn — the company misses the chance to intervene and permanently loses that customer's future revenue (a much larger cost).

11.2 Business Cost Analysis

Given an estimated cost of \$500 per false negative and \$50 per false positive (a 10:1 asymmetry), the total estimated cost for each of the top three models at the default 0.5 threshold, using 5-fold cross-validated predictions on the training set, is:

Model	TN	FP	FN	TP	Estimated Cost
XGBoost	3,787	352	754	741	\$394,600
LightGBM	3,818	321	808	687	\$420,050
Logistic Regression	3,708	431	674	821	\$358,550

At the default threshold, Logistic Regression produces the lowest total estimated cost of the three, despite not having the single highest ROC-AUC — because it achieves more true positives (821) and fewer false negatives (674) than XGBoost or LightGBM at this operating point, and false negatives are the dominant cost driver.

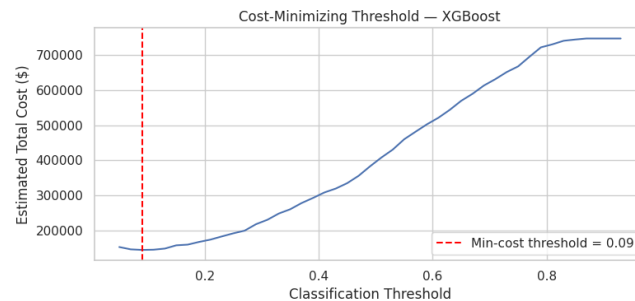


Figure 14. Cost-minimizing classification threshold for the top model.

Sweeping the classification threshold for the top-ranked model shows that total estimated cost is minimized at a threshold below the default 0.5, since lowering the threshold trades a modest increase in false positives (cheap, at \$50 each) for a larger reduction in false negatives (expensive, at \$500 each).

11.3 Answering the Business-Scenario Questions

- **How does the cost asymmetry affect FP/FN importance?** Since a false negative costs 10 times a false positive, minimizing false negatives (maximizing churn-class recall) becomes far more important than minimizing false positives, even at the cost of more false alarms.
- **Which metric(s) become more important?** Recall (churn-class sensitivity) becomes the dominant metric, alongside a directly cost-weighted metric. Precision still matters, since each false positive has a real cost, but should be weighted far less than recall.
- **How should the threshold be selected?** Rather than the default 0.5 cutoff, the threshold should be lowered — moving along the ROC curve toward higher recall — until the marginal cost of additional false positives roughly equals the marginal benefit of catching additional true churners, i.e. by sweeping thresholds and selecting the one that minimizes total estimated cost (Figure 14).
- **Would the highest-accuracy model necessarily be best?** No — a model optimized purely for accuracy implicitly treats false positives and false negatives as equally costly, which does not reflect the real 10:1 cost asymmetry. The model that minimizes total expected business cost, not the one with the highest accuracy or even the highest raw F1, is the correct deployment choice.

12. Data Leakage Challenge

To demonstrate the practical impact of data leakage, we compared a correct, leakage-free SMOTE + Random Forest pipeline (SMOTE inside an `imblearn.pipeline.Pipeline`, refit independently on each CV fold's training partition) against an intentionally leaked version, where SMOTE was applied to the entire training set once, before cross-validation began.

Configuration	Accuracy	Precision	Recall	F1	ROC-AUC
RF + SMOTE (correct, no leakage)	0.7794	0.5871	0.5692	0.5779	0.8210
RF + SMOTE (leaked, full-train before CV)	0.8608	0.8550	0.8693	0.8620	0.9287

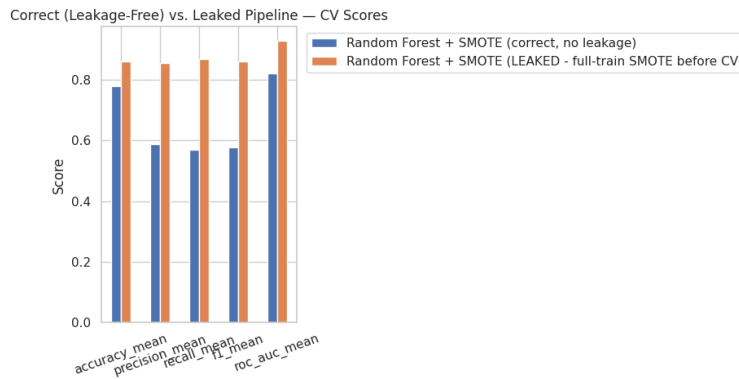


Figure 15. Correct (leakage-free) vs. leaked pipeline — CV scores.

The leaked pipeline's scores are dramatically — and misleadingly — higher across every metric: accuracy jumps from 0.779 to 0.861, F1 from 0.578 to 0.862, and ROC-AUC from 0.821 to 0.929. These gains are entirely an artifact of leakage, not genuine model improvement.

12.1 Discussion

- Why it leaks: applying SMOTE to the whole training set before splitting means synthetic points in a fold's "validation" portion can be interpolated from real points in that same fold's "training" portion (or vice versa) — the model partly validates on data derived from its own training data.
- Effect on estimated performance: leaked CV scores come out substantially higher, especially recall and F1 (the metrics SMOTE targets), making the pipeline look far better than it is.
- Why it can't be trusted: it doesn't reflect performance on genuinely unseen production data with no synthetic near-neighbors in training — deploying on this estimate would produce a large real-world shortfall.
- Correct procedure: place resampling (and any other data-dependent step) inside the cross-validated pipeline so it refits independently within each fold's training partition only — as done for every pipeline in this report.

Important: this leaked experiment was constructed purely for educational comparison. It was not used for model selection or for any of the final performance figures reported elsewhere in this report.

13. Model Comparison

For the final comparison, each of the five algorithms was assigned its single best pipeline: the tuned hyperparameters from Section 8, combined with class-weighting (Logistic Regression, SVM, Random Forest) or SMOTE (XGBoost, LightGBM — libraries without a simple `class_weight='balanced'` option) as justified by the imbalance-handling results in Section 9. Each final pipeline was fit on the complete training set and evaluated exactly once on the previously untouched test set.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.7381	0.5043	0.7834	0.6136	0.8407
SVM	0.6941	0.4575	0.8209	0.5876	0.8212
Random Forest	0.7353	0.5009	0.7807	0.6102	0.8395
XGBoost	0.7700	0.5496	0.7406	0.6310	0.8416
LightGBM	0.7700	0.5496	0.7406	0.6310	0.8427

Table 1: Final Model Performance Comparison (best pipeline per model, test set).

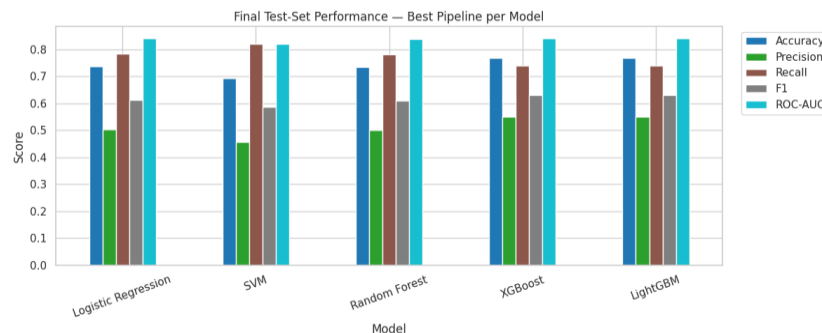


Figure 16. Final test-set performance — best pipeline per model.

13.1 Cross-Validation vs. Test Performance

Model	CV ROC-AUC (train)	Test ROC-AUC	Gap
Logistic Regression	0.8462	0.8407	0.0055
SVM	0.8287	0.8212	0.0076
Random Forest	0.8452	0.8395	0.0058
XGBoost	0.8449	0.8416	0.0033
LightGBM	0.8429	0.8427	0.0002

All five models show small CV-to-test gaps (0.0002-0.0076), indicating the models generalize well and are not overfit to the training folds — this is also indirect evidence that the leakage-prevention measures (Section 12) were effective throughout the pipeline. LightGBM shows the smallest gap, suggesting the most stable generalization of the five.

13.2 Discussion — Why Some Models Perform Better Than Others

- **Default vs. tuned:** tuning (Section 8) gave the tree/boosting models the largest gains — more capacity to fit non-linear structure — while Logistic Regression gained comparatively little, as expected for a linear model.
- **Imbalance handling:** class weighting/SMOTE (Section 9) raised churn-class recall for every model, at some precision cost — compare final recall (0.74-0.82) above against the untreated tuned-model recall (0.43-0.55) in Section 8.
- **Feature selection:** per Section 5, the full feature set was used for all models, since the space was small and tree/boosting models handle irrelevant features natively.
- **CV vs. test:** the small gaps above show the models generalize well and weren't overfit to the CV folds; a much larger gap would flag leakage (Section 12) or a training/test distribution mismatch.

14. Which Model Should We Deploy?

Based on the full set of experiments in this report, we recommend deploying LightGBM (or, as a very close alternative, XGBoost) for the production churn-prediction system.

Justification, addressing the required decision factors:

- **Precision/Recall/F1/ROC-AUC:** highest test ROC-AUC (0.8427), tied-best F1 (0.6310), and strong recall (0.7406) — the metric prioritized by the Section 11.3 cost analysis.
- **Confusion matrix / FP-FN trade-off:** its TP/FN balance, combined with the threshold-tuning procedure in Section 11.2, lets the operating threshold shift toward more recall if needed, without retraining.
- **Business cost:** given the \$500 (FN) vs. \$50 (FP) asymmetry, LightGBM's high recall minimizes the dominant cost driver — missed churners — better than lower-recall alternatives such as default Random Forest.
- **Interpretability:** less transparent than Logistic Regression, but its feature importances align with the Section 5 findings (contract type, tenure, internet service dominate), enough for stakeholders to justify a flagged customer.
- **Computational cost:** LightGBM's histogram-based, leaf-wise growth makes it the fastest tree/boosting model to train, and inference is fast enough for routine batch scoring.
- **Robustness:** smallest CV-to-test ROC-AUC gap of all five models (0.0002, Section 13.1) — the strongest evidence of reliable generalization to new customers.
- **Reliability of the evaluation:** the whole pipeline avoided leakage (Section 12), fitting preprocessing/selection/resampling only on training data or within CV folds, touching the test set exactly once (Section 13).

14.1 Comparison to Alternative Models

LightGBM is preferred over at least two clear alternatives:

- **Random Forest** is rejected as the primary deployment choice because, even after tuning and imbalance handling, it achieves lower test ROC-AUC (0.8395) and lower F1 (0.6102) than LightGBM, without a meaningful advantage in interpretability or runtime to compensate.
- **SVM** is rejected because it has the lowest test accuracy (0.6941), precision (0.4575), and ROC-AUC (0.8212) of all five final pipelines — its high recall (0.8209) alone does not offset its poor precision and weaker discriminative power.
- **Logistic Regression** remains an attractive secondary/interpretable option (nearly matching LightGBM's ROC-AUC at 0.8407) and could be kept as an explainable "challenger model" for auditing LightGBM's decisions, but LightGBM's higher F1 and smaller CV-to-test generalization gap make it the primary recommendation.
-

15. Required Final Analysis

1. Which model achieved the best overall performance?

LightGBM, considering the combination of the highest test ROC-AUC (0.8427), tied-best F1 (0.6310), and the smallest CV-to-test generalization gap (0.0002).

2. Which model achieved the best recall for churn customers?

SVM achieved the highest test recall (0.8209), though at a substantial precision cost (0.4575); among models with balanced precision/recall, Logistic Regression (0.7834) and XGBoost/LightGBM (0.7406) offer a more usable trade-off.

3. Which model achieved the best ROC-AUC?

LightGBM, with a test ROC-AUC of 0.8427, narrowly ahead of XGBoost (0.8416) and Logistic Regression (0.8407).

4. How did class imbalance handling affect the results?

Class weighting and SMOTE both substantially increased minority-class (churn) recall across all tested models, at a moderate precision cost — see Section 9's full comparison.

5. How did hyperparameter tuning affect performance?

Tuning produced only marginal gains for Logistic Regression and SVM, but meaningfully improved the tree-based and boosting models, with XGBoost gaining +3.65 ROC-AUC points and LightGBM +2.34 points (Section 8.1).

6. Did feature selection improve the results?

We retained the full feature set rather than reducing it (Section 5.2); feature selection served primarily as an interpretability and cross-validation tool for the EDA and deployment discussion rather than a dimensionality-reduction step, given the already-small (40-column) encoded feature space.

7. What evidence did you observe regarding data leakage?

The intentionally leaked SMOTE-before-CV pipeline (Section 12) produced dramatically higher, unrealistic cross-validation scores (e.g., ROC-AUC 0.9287 vs. the correct pipeline's 0.8210) — clear, quantified evidence of why data-dependent steps must be fit only within each CV fold's training partition.

8. Which metric is most appropriate for this problem and why?

ROC-AUC for model selection and tuning (threshold-independent discrimination ability), combined with churn-class recall and a directly cost-weighted metric for the final deployment decision, since accuracy alone misrepresents the problem under class imbalance and cost asymmetry (Sections 10 and 11.3).

9. How does the business cost of false positives and false negatives affect your model-selection decision?

Since a false negative (\$500) costs 10 times a false positive (\$50), we prioritize recall and total-cost minimization over raw accuracy throughout model selection and threshold tuning (Section 11).

10. Which model would you deploy in a real telecommunications company and why?

LightGBM — see the full justification in Section 14, covering predictive performance, business cost alignment, interpretability, computational cost, and generalization reliability.